Daniel Park
Professor Cowan
Project 1

Ship on Fire

Assignment Context:

In this assignment, I was asked to design an additional bot, in addition to bots 1-3, that can safely traverse a ship/maze and arrive at its goal of reaching the button, while safely avoiding a fire that has a chance to spread on each turn. The ship is randomly generated, and the bot, the button and fire are all placed randomly. Additionally, to ensure fairness, every bot plays the exact same game (same starting point, fire, and ship), to ensure that I was able to properly decipher the efficiency or win rate of each bot.

Bot 4 (Question 1)

Initial Brainstorming:

Bot 1 calculates the path to the button once and executes that plan ignorant of where the fire will spread. Bot 2 replans its path every turn to avoid current fire cells that have spread since the initial turn. Bot 3 executes a similar strategy but also avoids open cells that are directly adjacent to the fire, and, if that isn't available, it acts like bot 2. Knowing this, I wanted to build Bot 4 around these questions: "will I get to the button before the fire?" and "When/How will I know if I will get to the button before the fire?"

What Bot 4 Does:

Bot 4's algorithm is a combination of a Breadth First Search (*BFS*), Dijkstra's Cost Search Method, and A-Star method. Additionally, compared to Bots 1-3, Bot 4 is given the value of q to help make a more informed decision. At every turn a BFS starts from every burning cell, which is then used as a metric to calculate the amount of steps the fire is from each cell. So if a cell is 'd' steps from the fire, the fire would take around 'd/q' turns to reach that cell. Similarly, a second BFS is then executed from the button so Bot 4 knows the distance from the button from each cell. This is the heuristic (h) that A-Star utilizes.

Bot 4 then gives every step a cost or weight based on whether it would reach that cell before the fire. The lower the cost, the more the bot is encouraged to take that step and vice versa. This cost is produced through 4 formulas.

Formula 1: *Fire_time = fire_distance / q*

As mentioned earlier, this formula estimates how many turns until the fire reaches this cell. An illustration of this formula can be represented when $q = 0.3$ and a cell 3 steps from the fire, estimating that the cell will be caught in around 10 turns.

Formula 2: *ratio = t / fire_time*

This formula gives a ratio/percentage of the bot's arrival time compared to the fire, where "*t*" is the number of turns Bot 4 will reach a cell counting from the bot's current position. For example, a ratio of 0.5 would mean that Bot 4 gets there in about half the time the fire needs; a ratio of 1 would mean a tie; and anything greater than 1 would clearly indicate that the fire is expected to arrive at that cell before the bot.

Formula 3: *Risk* = {

0	*if* ratio < 0.5,
(ratio - 0.5)/0.5	*if* 0.5 <= ratio < 1,
1	*if* ratio >= 1

}

This formula utilizes Formula 2's ratio to give a risk score to indicate how dangerous it is to step onto a cell. The score ranges from 0 to 1, where 0 is considered safe and 1 is considered when the fire is expected to arrive first. At any ratio below 0.5, the bot gets to the cell in less than half the fire's expected time (d/q). The fire's actual arrival time varies from each game; but, the fire would have to be much faster than usual to beat the bot, which is why I treat this as safe (risk 0). At a ratio of 1 or more, the fire is expected to arrive first, so the risk is the highest at 1. Anything that lands between 0.5 and 1 is a closer race. To properly represent the even and consistent rise of risk, the formula of (ratio - 0.5) / 0.5 is then utilized. So a ratio of 0.75, halfway between 0.5 and 1, gives the proper risk score of 0.5.

Formula 4: *step cost = 1 + 10 x risk*

This is the formula that calculates the cost for Bot 4 to step onto a cell. The 1 in this formula represents the baseline cost. Every step takes a turn, and a step will always have at least a cost of 1. *10 x risk* represents the extra cost/weight for danger. 10 is the multiplier on how far out of its way that the bot will go to avoid a risky cell. This essentially means that one very risky cell will cost as much as 10 extra safe steps, so the bot will take a detour of up to 10 steps to avoid it. A smaller penalty makes the bot bolder, which then makes the bot behave like bot 2, and a larger penalty makes the bot avoid any risk even when the detour is huge. I simply picked 10 by hand as an intuitive middle ground.

A- Star Search:

A-Star is implemented as a min-heap priority queue to find the path from the bot to the button with the lowest total cost. The cell with the lowest '$f = g + h$' is explored next, where g is the total cost of the path so far and h is the BFS distance to the button.

As mentioned earlier, the heuristic (h) comes from the second BFS, the one that starts at the button. The button receives a h of 0, its neighbors 1, and so on. This BFS ignores the fire, and every step costs at least 1, allowing h to never guess higher than the actual/real cost of reaching the button. This makes h admissible.

G is the total cost of the path so far and where all the formulas I've mentioned come together. G starts at a value of 0 on the bot's current cell. Then every time A-Star considers stepping from a cell onto a neighbor, it runs each neighbor through all the formulas. Each neighbor then receives a value of f that consists of g (the current cell's value + the cost of that step) and h. All the open and nonburning neighbors are then added to the priority queue with their respective value of f. A-star keeps popping the cell with the smallest f until it pops the button, giving the entire path to the button. Bot 4 then only takes the first step of that path, and on the next turn the whole search runs again with the new fire.

It is important to mention that for the button, I use '*t-1*' instead of '*t*', as pressing the button ends the game before the fire spreads on that turn; and, secondly, cells that are currently burning are never entered, as stepping on them would lose the game.

Efficiency of Bot 4:

As mentioned, on each turn, Bot 4 runs two BFS searches, each search visiting every open cell once, and then an A-star search through the min heap priority queue. This is more work and memory compared to the other bots, which makes Bot 4 slower. To keep things efficient, the two BFS maps are built only once per turn and stored in their respective dictionaries. A-star can then look up a cell's fire distance and the distance to the button instantly instead of having to start another search. A-star also utilizes Python's heapq library to build the minheap priority queue instead of searching through a normal list to find the smallest value of *f*. Additionally, because this is a turn based game, the fire never spreads while the bot is thinking. The fire and bot take turns moving allowing the results of bot 4 to not be negatively affected.

How I Got Here/ Failures / Obstacles:

My initial version of Bot 4 also ran 2 BFS maps. The first BFS started from the fire to get each cell's distance from the fire, and the second BFS started from the bot and gave each cell a value of *t* (the number of turns the bot needed to get there). The bot then took a step onto a new cell, only if the value of "*t*" was less than the distance. If no route passed this rule, it simply took the shortest route. In these tests, this made this version of Bot 4 behave exactly the same as Bot 2, as the worst case assumption made the Bot too pessimistic. *D* counts the steps, but the fire doesn't always move one cell at every turn. This is when I realized that dividing the fire distance by *q* allows Bot 4 to answer the question "When will the fire approximately get to this cell?"

My second version of Bot 4 kept Bot 2's shortest path rule, but, if two moves were equally short, it picked the move that was further from the fire. This barely changed anything. The ship consists mostly of narrow corridors with only one way forward, so, most of the time, there was no choice for the bot to make. When there was a tie, both choices usually led to the same place, equally close to the fire. This version only ever considered the shortest routes, just like Bot 2. This told me that avoiding the fire helps the bot if the bot is allowed to take a longer route.

My third version of Bot 4 used the value of q. The bot used Formula 2 (ratio = t/ fire_time), and only allowed a cell if its ratio was below a certain cutoff. I used a list of six cutoffs (0.5, 0.75, 1, 1.5, 2, and 3). The bot started with the strictest cutoff of 0.5, which only allowed cells the bot reaches in less than half the time the fire needs, and then ran a BFS using only the allowed cells. If there was no route that reached the button, the cutoff loosened to the next value. This was the first time that this version performed better than the given bots, as it now took a longer route if the shorter route wasn't allowed. However, there were some issues. A cutoff of 0.75 treated a cell with a ratio of 0.74 as completely safe, while a ratio of 0.76 was forbidden. Additionally, the bot couldn't weigh out the options, as every allowed cell counted the same. This meant that among the allowed cells the bot simply took the shortest route. Another issue was that the results depended on the values of my cutoff. When I tried a list that started at the value of 1, the bot did worse. Lastly, because the bot loosened the cutoff one step at a time, it sometimes needed up to six searches of the whole ship in a single turn.

The current/final version of Bot 4 fixed these issues by first turning risk into a weight/cost instead of a simple yes or no rule. Rather than a cutoff, *Formula 3* is used as a gradual scale, so cells with ratios of 0.74 and 0.76 now get almost the same risk. The risk value is then fed into *Formula 4* so risky cells are more expensive. This now ultimately allows the bot to weigh its options, have no need for a cutoff list, and only need one search rather than up to six. Since steps also now have differing costs, BFS is no longer solely relied upon to find the best route. This is where the implementation of A-star comes in, adding up the costs of each step and using the BFS distance to the button to head in the right direction.

Results (Question 2)

Setup / How I Ran My Experiment:

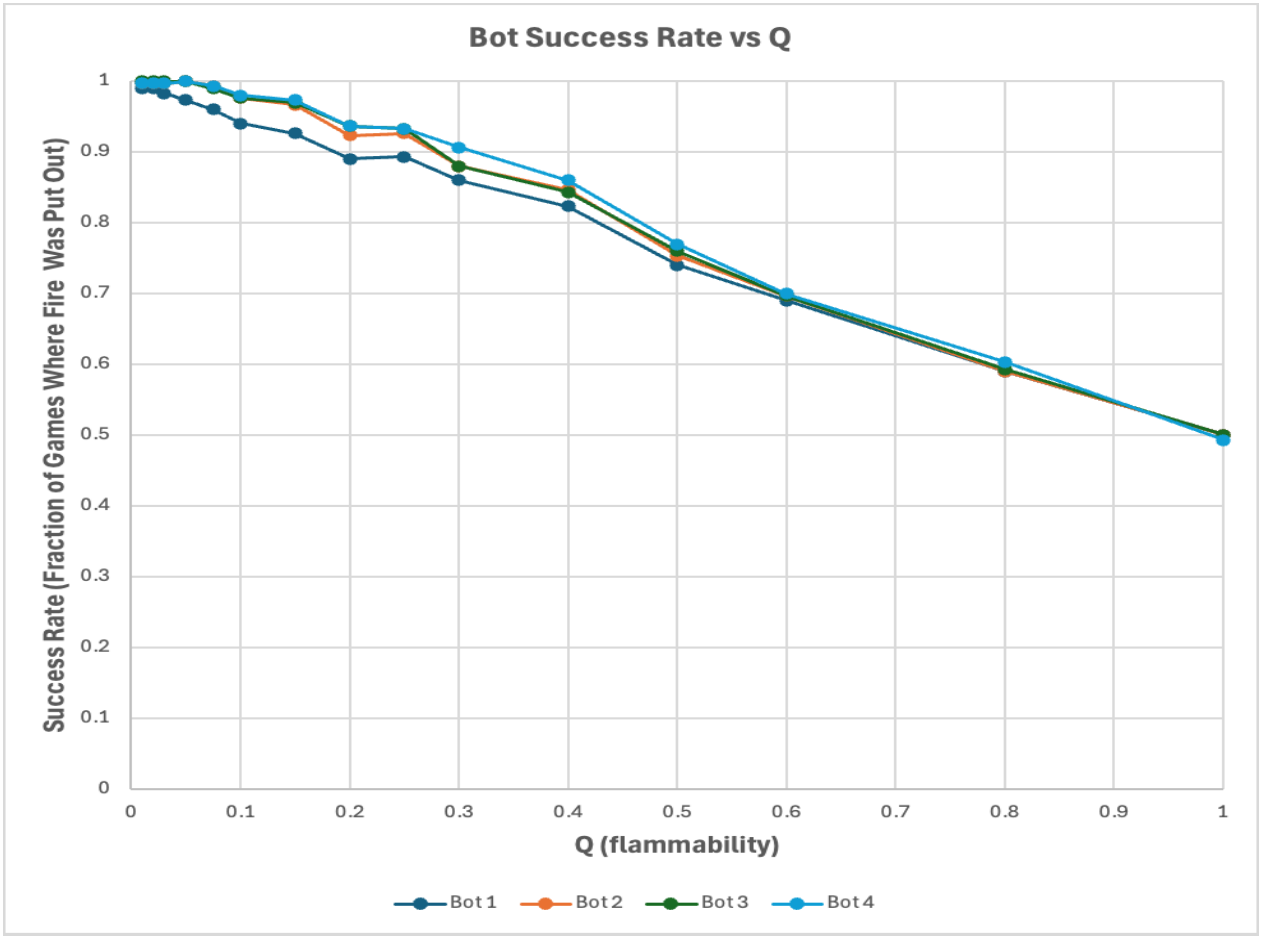
In my experiment, every ship is generated at 40 x 40. I decided to keep this as my consistent ship size, as this made each experiment take a reasonable time of 20-30 minutes to complete. When I tried running a larger ship, the terminal indicated that the approximate time for my experiments to complete would be closer to an hour. To mention briefly, despite risking my professional

"researcher" tone and a lower grade. Waiting an hour every time I encountered a bug or an error in my code was horror.

Additionally, I made 60 different ships and each bot played every ship 5 times. In each of those 5 games, the bot, fire, and button all start in different positions. This ultimately leads to exactly 300 games for each bot at every value of *q* (the fire's flammability). Moreover, to make sure every bot plays the exact same games, I used fixed random seeds (ship *i* uses seed *i*, game *n* on that ship uses seed ship x 1000 + *n*). This also means the same 300 games are used at every value of *q*.

I ran two experiments. The first experiment gives the full picture by testing 15 varying values of q from the minimum (0.01) to maximum (1.0). The second experiment tests 18 values of q between 0.04 and 0.5, as this is the "interesting range" where the performance of the bots differ the most. Each experiment saves its results to a CSV file, which were then utilized to make the graphs in Excel.

Graph 1:

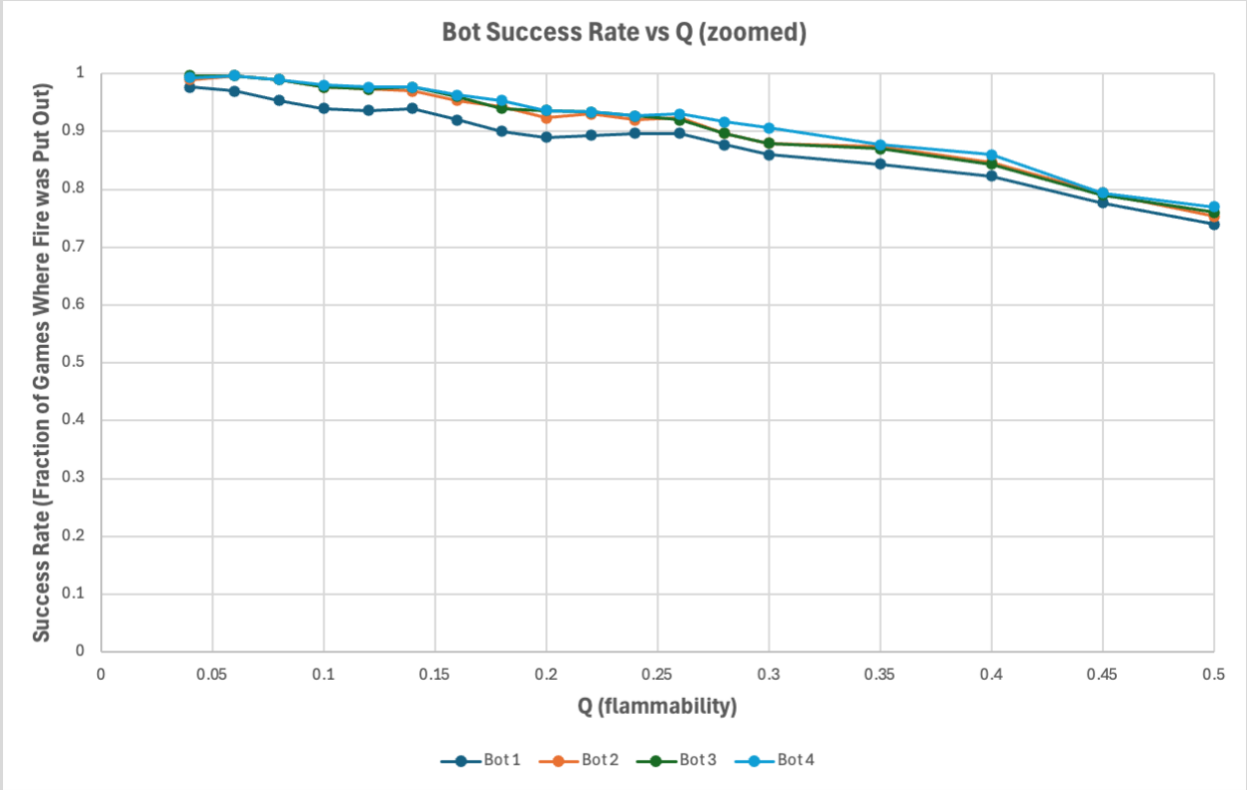


Graph 1 Explanation:

The success rate of each bot gradually decreases as the rate of the fire increases. When the fire is very slow (q below 0.1), Bots 2,3,4 win almost every game. Bot 1 is a bit lower as it never replans its path, so it sometimes walks into fire cells that spread onto its original path. However, starting at around $q = 0.6$, the performance of all bots starts to become the same. At q = 0.6, Bots 1,2,3,4 won 207, 209, 209, and 210 games respectively, and at q = 1 Bots 1,2,3 won 150 games while Bot 4 won 148 games.

This is because when the fire is this fast, the game is mostly decided by whether the bot starts closer to the button than the fire. This is ultimately why I ran another experiment within the "interesting range" for better insight.

Graph 2:



Graph 2 Explanation:

This graph now properly illustrates where Bot 4 is better than the other three. From $q = 0.1$ to $q = 0.5$, Bot 4 either tied or had the most wins. The biggest gap is apparent at $q = 0.3$, where Bot 4

won 272 out of the 300 games which is a 90.7% win rate, compared to 264 (88%) for Bots 2 and 3, and 258 (86%) for Bot 1. At the end of the range, $q = 0.5$, Bot 4 won 231 games, compared to 228 for Bot 3, 226 for Bot 2, and 222 for Bot 1. After $q = 0.5$ the fire starts becoming too fast where the difference between the Bots starts to increasingly dwindle. However, this "interesting range" is the range where fire is slow enough so the bots have enough time to take a safer detour, but fast enough that the shortest route is often dangerous.

While these differences are certainly small, with only a difference of a few games out of 300, Bot 4 was at the top or tied at every point from q = 0.1 to 0.5 (15 values of q in a row). Because every bot plays the exact same ships, starting positions, and fire, the difference comes from the bots' decisions rather than from Bot 4 getting easier games. This consistency makes me believe that this wasn't "sheer luck", although running more games at each q would make the difference even more certain.

Why Do the Bots Fail? (Question 3)

Was There a Better Decision?:

Every bot plays the exact same games with the same fire. This makes it easy for me to compare them. If one bot wins that another bot loses, then it clearly means that the losing bot made a worse decision somewhere while the winning bot made a better decision. For example, at $q = 0.3$, Bot 4 won 272 of the 300 games, compared to 264 for Bots 2 and 3 and 258 for Bot 1. Since the games are identical, those extra wins show that the bots really do make different decisions. This is exactly what caught my initial version of bot 4, which made the same moves as Bot 2. Moreover, focusing more on Bots 2 and 3, both bots almost always win the same number of games at every q. I believe this shows that Bot 3's extra step of avoiding cells next to the fire is insignificant, as it rarely changes the outcome.

Why Bot 1 Fails:

Bot 1 plans its path only once at the start and then never analyzes the fire again. So, when the fire spreads onto its planned route, the bot walks straight into the fire. A better decision definitely existed in the games that Bot 1 lost. By simply replanning its route at every turn, which is Bot 2's

logic, Bot 1 could have won more games. Bot 2 won 6 more games than Bot 1 at $q = 0.3$ (264 vs 258) just by doing that.

Why Bots 2 and 3 Fail:

Bots 2 and 3 are more sophisticated, as they never step onto a burning cell. This means that when they lose, it's because the fire caught up to them. The problem with these bots is that they only look at where the fire is right now, not where it will be. This makes Bot 2 content with taking a route right next to the fire. Bot 3 stays one additional cell away from the fire, but this still doesn't ask how fast the fire is spreading. Once the fire cuts the bots off, the bot doesn't have a path to win, ultimately making it a failure. The better decision would be to simply choose the route that is farther from the fire before getting surrounded. This is what my Bot 4 does, and at $q = 0.3$ it won 8 more games than Bots 2 and 3 (272 vs 264).

Why Bot 4 Still Fails:

Bot 4 still lost 28 of the 300 games at $q = 0.3$. Some of these were likely impossible for any bot to win, but there are certain weaknesses that my Bot 4 has.

The Fire Can Arrive Earlier Than the Average: Formula 1 (d / q) gives the fire's estimated arrival time. For instance, at $q = 0.3$, a cell 2 steps from the fire is estimated to be roughly 6.7 turns away. This makes Bot 4 treat this cell as safe if it can get there within 3 turns. So Bot 4 can choose a route that passes close to the fire, if the ratio *(Formula 2: t/ fire_time)* is less than 0.5. This gives the cell a risk of zero, but the Bot can still get unlucky if the fire spreads faster than average and reaches the cell first. I believe a better decision would have been to treat the cells that are very close to the fire with an additional cost, even when the ratio is below 0.5. With my current formulas, a cell right next to the fire can cost the same as a perfectly safe cell. Adding this additional cost to cells close to the fire and with ratios below 0.5 should give an even more accurate picture of the risk allowing the bot to make a better decision.

Too Careful When the Fire is Fast: When $q = 1$, Bot 4 won 148 games while Bots 1,2,3 each won 150 out of the 300 games *(Graph 1)*. This result initially made no sense to me, as Bot 4 is the most logically advanced bot compared to the others. However, I believe this is the reason. At $q = 1$, the

fire spreads every turn, so the bot has no time to take a detour. Bots 1,2, and 3 effectively take the shortest path, while Bot 4 can be pulled into a detour to stay away from the fire. But by the time the bot follows its longer detour route and starts making its way to the button, there are no possible routes left, as the fire has been spreading every turn.
I believe a better decision would have been to check whether a guaranteed safe path exists, one where the bot stays ahead of even a $q = 1$ fire (described in Question 4), and if it does, the bot just takes it.

The Ideal Bot (Question 4)

What Information Would It Use, Compute and How?:

I believe my Bot 4 used most of the available information that allowed it to make a more informed decision than the given bots. However, on top of the information it used, I believe the ideal bot would first check whether the game is an easy win (a guaranteed-safe route). The bot would assume the worst possible fire and then check whether a guaranteed-safe route exists. A BFS search from the starting position of the fire would be run, and the distance the BFS gives to each cell would be the earliest turn the fire could possibly reach that cell. Then a second BFS could be run from the bot. If the bot's distance to a cell is less than the fire's distance to that cell, the bot can step on it. Furthermore, if this second BFS reaches the button, this means that there is a guaranteed safe route that the Bot can take.

If no guaranteed-safe route exists, the ideal bot would need to use risk calculations like Bot 4, but more accurately. Instead of utilizing the fire's average time (d/q), the ideal bot would calculate the actual probability that the fire reaches each cell before the bot. This fixes the weakness of my bot 4 design where cells right next to the fire were sometimes treated as completely safe if the ratio was less than 0.5.

Lastly, I believe the ideal bot should make a break for the button, if the bot can stay ahead of even the fastest possible fire the entire way. This allows the bot to follow the guaranteed safe route without having to worry about the fire blocking the next cell the bot wants to take. Additionally, I believe another instance where the bot should make a break for the button is if every possible route is risky. In this particular case, more thinking won't find a safe route, so the bot should take the

shortest possible route. As mentioned before, one of the main shortcomings of my bot 4 was that when q = 1, the bot took a detour instead of the shortest path, even though the shortest path would have won.

Computation Against Intelligence:

Bot 4 does a lot more work compared to the other bots. It runs 2 BFS searches and an A-star search every turn. The extra computation doesn't hurt in this project, as the project is a turn based game so the fire and the bot wait until the other has made a move. In an actual fire simulation, the bot wouldn't have the luxury to compute as long as it needs, as the fire would be spreading while it thinks.

Thus, building upon the ideas that I have already discussed, here's how I would balance the ideal bot. If the game is an easy win, the bot shouldn't compute anything else and just make a break for it. At a low and high q, the bot should only compute a replan through a BFS search every turn. The reasoning is at a low q, the fire is slow enough to the point where it didn't really matter how advanced my bot 4 design was as bots 2,3,4 pretty much won all the games. At a high q, taking the shortest possible route gave the bot the best chance of beating the fire to the button. At a medium q, roughly 0.1 to 0.5, the bot should have a risk calculation method like Bot 4, as this is where Bot 4 actually won more games than the other bots (Graph 2).

Final Thoughts:

Conducting these experiments was an enjoyable experience. The trial and error of trying to come up with an algorithm for Bot 4 was definitely the most challenging, while finally seeing how Bot 4 compared to Bots 1,2, and 3 was the most interesting. On a more general level, as a person, it's easy to look at the grid and quickly mention where and how the ideal bot should move. However, trying to actually code, describe, and explain the thinking behind these moves quickly brought me down to reality. Things that feel obvious to a person, like "don't walk right next to the fire" or "just go for it, there's no time" now had to be converted into exact rules and numbers for the bot to comprehend. Additionally, many of the rules I picked had side effects I didn't expect, like my first version of Bot 4 behaving exactly like Bot 2, and my final Bot 4 being too careful at q = 1. This project ultimately opened my eyes to how I can accurately articulate my thought processes for a bot to properly understand.